

Colecciones documentales: gestión y consultas

2. Colecciones documentales: gestión y consultas

La comparación relacional/documental y el porqué de usar dos motores a la vez ya los has visto en el apartado anterior. Hoy ves la creación explícita de una colección (y su contrapartida, eliminarla), y una herramienta nueva para cuando una consulta ya no cabe cómodamente en el nombre de un método.

Colecciones: crear, eliminar, listar

Todo esto se hace desde mongosh, no desde tu código Java. Con PostgreSQL tienes a Hibernate de tu lado: ddl-auto puede crear la tabla por ti, sin que escribas ni un CREATE TABLE. Spring Data MongoDB no convierte automáticamente tus anotaciones de Bean Validation en reglas de validación de la colección — son dos capas independientes.

En este apartado vas a configurar el validador directamente desde mongosh, para ver que la regla vive también en la propia base de datos.

El hueco: MongoDB no valida la forma de un documento

Ya sabes que MongoDB no valida claves foráneas. Pero hay un segundo hueco, más básico: por defecto, tampoco valida la forma de un documento. Nada te impide, desde mongosh, insertar esto directamente:

```
db.nota_lectura.insertOne({ libroId: 1, autor: "ana", puntuacion: 999 })
```

puntuacion: 999 no tiene ningún sentido — tu DTO en Java, con `@Min(1) @Max(10)`, nunca dejaría pasar esto por tu API. Pero esa validación vive solo en tu código Java: cualquiera con acceso directo a Mongo se la salta sin esfuerzo, porque la base de datos no sabe que esa regla existe.

Cerrar el hueco: un validador en la colección

MongoDB deja poner un validador directamente en la colección. En esta práctica, la opción más sencilla: eliminar la colección de pruebas y crearla explícitamente desde el principio, con sus reglas:

```
db.createCollection("nota_lectura", {
  validator: {
    $jsonSchema: {
      required: ["libroId", "autor", "puntuacion"],
      properties: {
        puntuacion: {
          bsonType: "int",
          minimum: 1,
          maximum: 10,
          description: "debe ser un número entero entre 1 y 10"
        }
      }
    }
  }
})
```

El validador en acción, y las otras dos operaciones

Con esto montado, el mismo insertOne de antes (puntuacion: 999) falla directamente en MongoDB, sin llegar nunca a tu aplicación — la base de datos rechaza el documento por sí sola, igual que rechazaría un INSERT en PostgreSQL que violara una restricción CHECK.

```
// Eliminar la colección entera
db.nota_lectura.drop()

// Listar qué colecciones existen en la base de datos
show collections
```

El mismo objetivo que un CHECK, con otra sintaxis

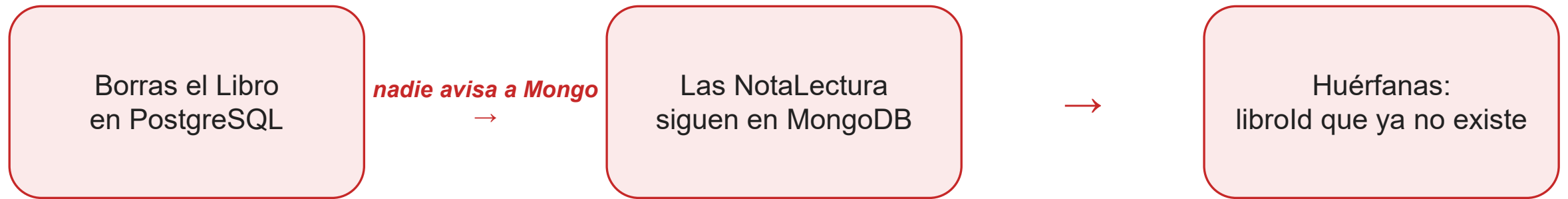
minimum: 1 y maximum: 10 cumplen un papel similar a un CHECK (puntuacion BETWEEN 1 AND 10) de PostgreSQL, mientras que incluir puntuacion en required obliga a que el campo exista, de forma comparable a un NOT NULL. En ambos casos, la idea es la misma: que la propia base de datos también proteja la validez de los datos, sin depender únicamente de la aplicación.

Eliminar documentos en bloque

Borrar la colección entera con `drop()` es un martillo demasiado grande para un problema más concreto: eliminar solo algunos documentos, los que cumplen una condición, sin tocar el resto de la colección.

Imagina que borras un Libro de PostgreSQL: sus notas de lectura en MongoDB no se enteran de nada — son dos motores completamente independientes, sin ninguna relación real entre ellos.

Notas de lectura huérfanas



Se quedan ahí, huérfanas, apuntando a un libroid que ya no existe.

deleteByLibroId, y quién lo llama

```
public interface NotaLecturaRepository
    extends MongoRepository<NotaLectura, String> {
    long deleteByLibroId(Long libroId);
}
```

Por sí solo, este método no hace nada: alguien tiene que llamarlo cuando se produzca el borrado del libro. El servicio que elimina el libro publica un evento en RabbitMQ y un consumer lo recibe de forma asíncrona para llamar a `deleteByLibroId` — durante un breve intervalo pueden existir notas huérfanas, hasta que el consumer procese el evento.

Cuando el naming ya no basta

findByLibroId, deleteByLibroId... el naming de Spring Data funciona muy bien con una o dos condiciones. Con dos, todavía se lee bien:

```
List<NotaLectura> findByLibroIdAndPuntuacionGreaterThanOrEqualTo(  
    Long libroId, Integer puntuacionMinima);
```

Añade una tercera condición y mira qué le pasa al nombre del método.

Con una tercera condición, el nombre se dispara

```
List<NotaLectura> findByLibroIdAndPuntuacionGreaterThanOrEqualTo  
    AndComentarioIsNotNull(  
        Long libroId, Integer puntuacionMinima);
```

En algún punto, el nombre del método deja de ser más claro que la propia consulta.

@Query: el filtro, no el nombre

```
public interface NotaLecturaRepository
    extends MongoRepository<NotaLectura, String> {

    @Query("{ 'libroId': ?0, 'puntuacion': { $gte: ?1 },",
           "'comentario': { $ne: null } }")
    List<NotaLectura> buscarBuenasNotasConComentario(
        Long libroId, Integer puntuacionMinima);
}
```

?0, ?1... son marcadores de posición para los parámetros del método, en el orden en que los declaras — Spring Data los sustituye por ti antes de mandar la consulta a Mongo.

El mismo mecanismo de JPA, con otro idioma

@Query en Spring Data MongoDB cumple exactamente el mismo papel que @Query en Spring Data JPA (Tema 1): una vía de escape para cuando el naming se queda corto o se vuelve ilegible. Solo cambia el idioma dentro de las comillas — JPQL o SQL nativo allí, un documento de filtro de Mongo aquí.

El límite de @Query: filtros opcionales

Pero hasta @Query tiene un límite: el filtro sigue siendo fijo, escrito de antemano — solo los valores cambian, no las condiciones en sí.

Imagina una búsqueda con varios criterios, todos opcionales, donde no sabes de antemano cuáles va a rellenar quien la use. Para eso ni el naming ni @Query sirven bien.

Criteria y Query: el equivalente a Specification

```
@Service
@RequiredArgsConstructor
public class NotaLecturaService {

    private final MongoTemplate mongoTemplate;

    public List<NotaLectura> buscar(
        Long libroId, String autor, Integer puntuacionMinima) {
        Query query = new Query();

        if (libroId != null) {
            query.addCriteria(Criteria.where("libroId").is(libroId));
        }
        if (autor != null) {
            query.addCriteria(Criteria.where("autor").is(autor));
        }
        if (puntuacionMinima != null) {
            query.addCriteria(Criteria.where("puntuacion").gte(puntuacionMinima));
        }

        return mongoTemplate.find(query, NotaLectura.class);
    }
}
```


Cómo se combinan los filtros

MongoDB tiene su propio equivalente a las Specification que ya conoces de PostgreSQL (Tema 2): las clases Criteria y Query, que se combinan condición a condición, en tiempo de ejecución, añadiendo solo los filtros que de verdad han llegado.

Cada if añade una condición solo si ese criterio ha llegado — igual que ibas encadenando condiciones a una Specification con `.and(...)`, aquí vas añadiendo Criteria a la misma Query. No se practica en las prácticas de este módulo, pero conviene que sepas que existe.

- Por defecto, MongoDB no valida la forma de un documento — una colección se crea implícitamente al primer insert, sin ninguna regla. La validación puede definirse con `createCollection` y `$jsonSchema`, o añadirse luego con `collMod`.
- Borrar un Libro no borra automáticamente sus notas de lectura en Mongo, porque son dos motores independientes. `deleteByLibroId` elimina en bloque por naming; en la práctica, un consumer asíncrono lo llama al recibir el evento de borrado — la consistencia entre ambos motores es eventual.

Ideas clave (cont.)

- `@Query`, con sintaxis de filtro de Mongo, es la vía de escape cuando el naming se vuelve largo o ilegible — el mismo papel que `@Query` en Spring Data JPA, con otro idioma dentro de las comillas.
- Para filtros opcionales combinables en tiempo de ejecución, `Criteria/Query` con `MongoTemplate` es el equivalente Mongo de `Specification` — no se practica en este módulo, pero existe para cuando haga falta.